aivancity

La Grande Ecole de l'IA & de la Data

Paris Île-de-France · Nice Côte d'Azur

Novembre 2026

AI-Assisted Coding

De l'autocomplétion aux agents : coder plus vite, vérifier plus fort

Accueil

À propos

Head of AI @Koneetiv
Head of AI @Devoteam
Principal AI Architect @Chanel
Lead Data Scientist @SQLI
Senior Data Scientist @Schneider Electric
Data Scientist @Schneider Electric



Accueil

Ce que vous allez apprendre

À la fin du Jour 2, vous saurez…

Comprendre

Comment les LLM génèrent du code, pourquoi ils se trompent, et ce que dit la recherche sur la productivité.

Maîtriser Copilot

Complétions, chat, modes Plan et Agent, contexte, fichiers d'instructions. Et un aperçu de Claude Code.

Prompter pour coder

Écrire des prompts et des specs qui donnent du code correct au 1er ou 2e essai.

Vérifier

Détecter hallucinations, bugs et failles avec des tests, des revues et des outils.

Piloter des agents

Suivre un workflow guidé par la spec : plan, petites étapes, tests, commits.

Livrer un produit

Construire et présenter une app qui marche, en équipe, en une journée.

Accueil

Le cours en un coup d'œil

Apprendre en faisant : la plupart du temps, vos mains sont sur le clavier

12h

2 jours de 6h, 9h30–16h45

~75%

du temps en pratique : TP et projet

4

TP guidés le Jour 1

1

projet en équipe + démo le Jour 2

Jour 1 = apprendre et pratiquer le workflow. Jour 2 = l'appliquer pour de vrai sur votre produit.

Jour 1

Jour 1 : apprendre et pratiquer

Matin

09:30–09:45	Accueil & brise-glace
09:45–10:25	Partie 1 · Comment l'IA code
10:25–10:55	TP 0 · Installer Copilot
10:55–11:05	Pause
11:05–11:35	Partie 2 · Prompter pour coder
11:35–12:30	TP 1 · Prompt battle

Après-midi

13:45–14:10	Partie 3 · Faire confiance, vérifier
14:10–14:45	TP 2 · Chasse aux bugs
14:45–15:10	Partie 4 · Agents + Claude Code
15:10–16:10	TP 3 · De la spec à l'API (pause incluse)
16:10–16:25	Partie 5 · Usage responsable + quiz
16:25–16:45	Lancement du projet

Théorie · TP · Projet · Pause

Jour 2

Jour 2 : construire et présenter

Matin

09:30–09:45	**Stand-up & règles du jour**
09:45–11:00	**Sprint 1 · Squelette + 1re feature**
11:00–12:30	**Sprint 2 · Features + tests** point mentor, 5 min/équipe
12:30–13:45	**Déjeuner**

Après-midi

13:45–15:00	**Sprint 3 · Consolidation** tests, README, AI_LOG
15:00–15:15	**Gel du code & prépa démo**
15:15–16:30	**Démos : 8 min + 2 min de questions**
16:30–16:45	**Votes, rétro, clôture**

Rituels · Développement · Démos · Pause

Accueil

Comment on travaille ensemble

Votre poste

VS Code, Git, Python 3.11+, un compte GitHub. Copilot s'installe au TP 0.

Binômes, puis équipes

TP du Jour 1 en binôme : Driver au clavier, Navigator relit. On échange à chaque TP.

Interactif

Brise-glace : d'où partez-vous ?



À main levée

Levez la main, levez-vous ou montrez vos doigts

1. **Main levée : qui a déjà utilisé l'IA pour coder ?**
   Puis : qui l'utilise tous les jours ? Gardez la main levée.

2. **À voix haute : quel outil utilisez-vous ?**
   ChatGPT · Copilot · Cursor · Claude · …

3. **« J'ai confiance dans le code généré par l'IA » : montrez 1 à 5 doigts**

4. **Devinez : des devs expérimentés avec IA, dans une étude de 2025, étaient…**
   Main levée pour A : 20 % plus rapides · B : aussi rapides · C : 19 % plus lents

Jour 1 · Partie 1

01

Comment l'IA écrit du code

De l'autocomplétion aux agents autonomes

Dans cette partie

1. 5 niveaux d'assistance IA
2. Ce que le modèle voit vraiment
3. Le paysage des outils
4. Ce que dit la recherche
5. Forces et angles morts

40 min

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · Partie 1

De l'autocomplétion aux agents

Cinq niveaux d'autonomie : plus d'autonomie, plus de vérification



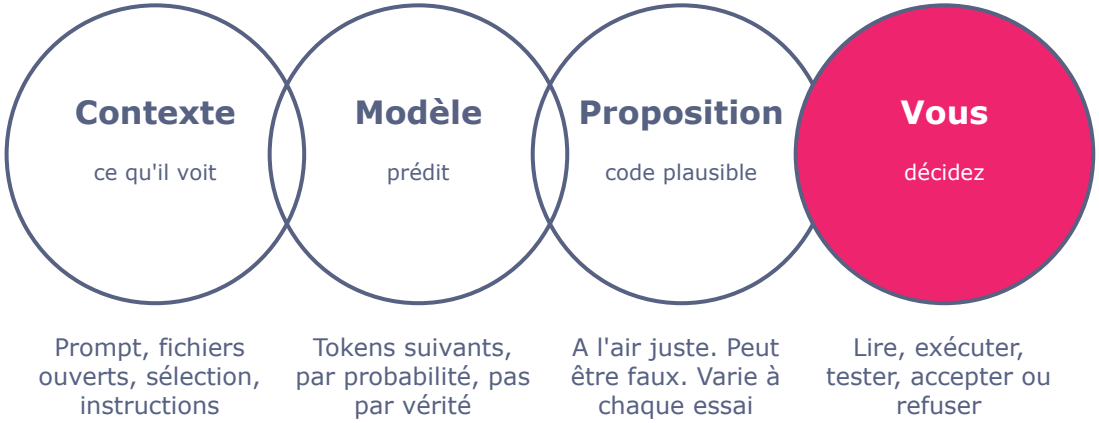
Compléter	Chat	Éditer	Agent	Agent cloud
Propose les lignes suivantes pendant que vous tapez. Vous faites Tab.	Répond, explique, rédige du code que vous copiez.	Modifie un ou plusieurs fichiers sur demande. Vous relisez le diff.	Planifie, modifie, lance commandes et tests, itère jusqu'au bout.	Travaille seul sur une issue dans le cloud et ouvre une pull request.

À chaque niveau, vous restez l'ingénieur responsable.

Jour 1 · Partie 1

Ce qui se passe quand vous faites Tab

Un LLM prédit les tokens suivants les plus probables. Il n'exécute pas votre code.



Jour 1 · Partie 1

Le contexte fait tout

Garbage in, garbage out : ce que vous donnez au modèle décide du résultat

Fichiers ouverts

Copilot lit le fichier courant et les onglets liés. Fermez le bruit, ouvrez l'utile.

Instructions

Règles projet dans copilot-instructions.md ou CLAUDE.md, appliquées à chaque requête.

Références explicites

#file, #codebase, une sélection, un message d'erreur, une page de doc.

Exemples

Des paires entrée/sortie et du code existant dans votre style valent mieux que de longues explications.

Sortie des outils

Pour les agents : résultats de tests, logs et stack traces nourrissent l'étape suivante.

Limites

La fenêtre est limitée : les longs chats dérivent. Un nouveau chat par tâche.

Jour 1 · Partie 1

Le paysage des outils

Beaucoup d'outils, quatre familles. On se concentre sur Copilot, avec un aperçu de Claude Code

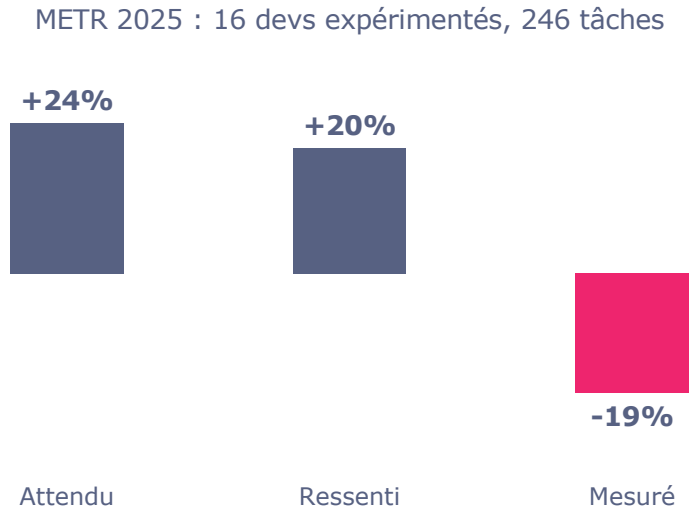
Famille	Exemples	Où ça tourne	Idéal pour
Assistants d'IDE	GitHub Copilot, Cursor, Windsurf	Dans l'éditeur	Le code au quotidien
Agents terminal	Claude Code, Codex CLI, Gemini CLI	Terminal, sur votre repo	Tâches multi-fichiers, refactors
Agents cloud	Agent de code Copilot, Codex cloud	Dans le cloud, via issues/PR	Tâches de backlog bien cadrées
Générateurs d'apps	Lovable, Bolt, v0	Dans le navigateur	Prototypes, maquettes

Les outils changent chaque mois. Le workflow appris aujourd'hui reste.

Jour 1 · Partie 1

Ce que dit la recherche

Adoption massive, confiance limitée, et productivité pas automatique



84% — **des développeurs utilisent ou vont utiliser des outils IA**

46% — **se méfient de la justesse des réponses IA (33 % leur font confiance)**

66% — **citent en n°1 les réponses « presque justes, mais pas tout à fait »**

Sources : METR, juillet 2025 · Stack Overflow Developer Survey 2025

Jour 1 · Partie 1

Forces et angles morts

Là où l'IA brille

- Code répétitif, CRUD, fichiers de config
- Tests, docstrings, brouillons de README
- Expliquer du code ou des erreurs inconnus
- Traduire entre langages et bibliothèques
- Regex, SQL, commandes shell
- Prototypes et premières versions

Là où elle trébuche

- Versions récentes ou rares des bibliothèques
- Règles métier jamais écrites
- Gros codebases et conception transverse
- La sécurité par défaut
- Cas limites : vide, zéro, dernier élément, fuseaux
- Savoir ce qu'elle ne sait pas

Jour 1 · TP 0

Installer GitHub Copilot

Tout le monde repart avec un assistant qui marche

30 min · individuel

Vous allez

1. Activer Copilot via GitHub Education
2. Installer l'extension VS Code
3. Découvrir l'interface et les raccourcis
4. Valider la checklist

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · TP 0

Obtenir Copilot gratuitement en tant qu'étudiant

1. **Créez ou connectez votre compte GitHub**
   Ajoutez votre email école dans Settings → Emails.

2. **Faites la demande GitHub Education**
   github.com/settings/education/benefits → Start an application

3. **Activez Copilot Student**
   Page Benefits → Learn more → suivez les étapes. Ne payez jamais.

4. **Installez VS Code + l'extension GitHub Copilot**
   Extensions (Ctrl+Shift+X) → « GitHub Copilot » → Installer.

5. **Connectez GitHub dans VS Code**
   Icône Comptes (en bas à gauche) → Sign in with GitHub.

6. **Dites bonjour**
   Ouvrez le Chat (Ctrl+Alt+I) et demandez : « Explique ce que tu peux faire dans ce workspace ».

Si ce n'est pas encore actif

- La vérification peut prendre plusieurs jours : faites-la avant le cours.
- Seules des offres payantes ? N'achetez rien. Attendez et réessayez.
- En attendant, utilisez le plan Copilot Free (quotas mensuels).
- Toujours bloqué : travaillez avec un voisin.

Jour 1 · TP 0

Votre cockpit Copilot

Apprenez-les par cœur aujourd'hui : vous les utiliserez 200 fois

Action	Windows / Linux	macOS	Pour quoi faire
Accepter / refuser une suggestion	Tab / Esc	Tab / Esc	Texte grisé pendant la frappe
Chat en ligne	Ctrl+I	Cmd+I	Modifier le code là où vous êtes
Vue Chat	Ctrl+Alt+I	Ctrl+Cmd+I	Modes Ask, Plan, Agent
Ajouter du contexte	#	#	#file, #codebase, #selection…
Commandes	/	/	/explain /fix /tests /doc /plan
Choix du modèle	Zone de saisie	Zone de saisie	Gardez Auto avec le plan Student

Jour 1 · TP 0

Checklist : êtes-vous prêts ?

Levez la main quand les 6 sont au vert

- **Icône Copilot active dans la barre d'état**
- **Le texte grisé apparaît dans un fichier .py**
- **Le chat répond en mode Ask**
- **Le mode Agent sait créer un fichier**
- **Kit du cours cloné et ouvert**
  dossiers lab1, lab2, lab3, project
- **pytest se lance dans le terminal**
  python -m pytest --version

Fini en avance ? Aidez un voisin : vous devenez mentor Copilot.

Jour 1 · TP 0

Bonus : Claude Code, un agent dans votre terminal

Démo de l'intervenant aujourd'hui ; testez si vous avez un accès

1. **Installer**
   macOS, Linux, WSL : curl -fsSL https://claude.ai/install.sh | bash

2. **Installer sous Windows (PowerShell)**
   irm https://claude.ai/install.ps1 | iex

3. **Compte**
   Nécessite un compte Claude Pro, Max, Team, Enterprise ou Console.
   Pas dans le plan gratuit.

4. **Démarrer dans votre repo**
   cd mon-projet && claude, puis /init pour générer CLAUDE.md

Gestes clés

```
/init    créer CLAUDE.md
Shift+Tab  mode plan
Échap  interrompre
/clear  nouvelle tâche
/compact réduire le contexte

« Planifie d'abord,
 puis fais l'étape 1
 et lance pytest »
```

Jour 1 · Partie 2

02

Parler à votre copilote

Des prompts et des specs qui produisent du code correct

30 min

Dans cette partie

1. Le prompt CRAFT
2. Mauvais vs bon prompt
3. Techniques Copilot
4. Fichiers d'instructions
5. Ask, Plan ou Agent ?

Jour 1 · Partie 2

Le prompt CRAFT

Cinq ingrédients d'un prompt qui donne du code qui marche

C

Contexte

Projet, langage, bibliothèques, fichier et fonction concernés.

R

Exigences

Ce que ça doit faire, étape par étape. Entrées et sorties.

A

Acceptation

Critères d'acceptation : cas limites, erreurs à lever, limites de perf.

F

Format

Signature, style, pas de nouvelle dépendance, forme de la réponse.

T

Tests

3–5 exemples entrée → sortie, ou les tests à passer.

Une checklist, pas un rituel : sautez ce que le fichier ouvert dit déjà.

Jour 1 · Partie 2

Même demande, deux prompts

Prompt vague : le modèle comble les trous en devinant. Décimales ? Négatifs ? Erreurs ?

Prompt CRAFT : chaque ambiguïté est tranchée par vous, et les exemples servent de tests.

› Vous mesurerez la différence au TP 1.

Vague

```
écris une fonction pour parser des durées
```

CRAFT

```
# Python 3.11, sans dépendance. Dans duration.py
# parse_duration(text: str) -> int (secondes)
# Unités d,h,m,s, dans cet ordre, une fois.
# Insensible à la casse, espaces autorisés.
# ValueError si : vide, unité inconnue,
# décimales, négatifs, mauvais ordre.
# "1h30m" -> 5400   "45s" -> 45
# "1d2h3m4s" -> 93784   "30m1h" -> ValueError
```

Jour 1 · Partie 2

Les techniques Copilot qui marchent

La signature d'abord

Écrivez nom, types et docstring ; laissez Copilot écrire le corps.

Découper

Une fonction, un prompt. Les grosses demandes donnent de grosses erreurs.

Pointer le contexte

#file, #codebase, code sélectionné, le message d'erreur exact.

Commandes slash

/explain pour apprendre, /fix sur une erreur, /tests pour une 1re suite, /doc.

Itérer, pas recommencer

« Bien, gère maintenant l'entrée vide » vaut mieux qu'un prompt géant réécrit.

Faites-la questionner

« Avant de coder, liste tes questions et hypothèses. »

Jour 1 · Partie 2

Écrire les règles une fois : les fichiers d'instructions

Copilot: .github/copilot-instructions.md

Claude Code: CLAUDE.md (généré avec /init)

Beaucoup d'agents : AGENTS.md

- Ajouté automatiquement à chaque requête. Court, concret et versionné avec le code.
- Modèle dans project/templates/.

.github/copilot-instructions.md

```
## Stack
- Python 3.11, FastAPI, pydantic v2, pytest
- Pas de dépendance sans demander d'abord

## Style
- Type hints, petites fonctions, docstrings
- Secrets via variables d'env uniquement
- SQL uniquement avec requêtes paramétrées

## Workflow
- Lire SPEC.md avant toute modification
- Proposer un plan, attendre validation
- Lancer pytest -q après chaque changement
- Ne jamais supprimer ni affaiblir un test
```

Jour 1 · Partie 2

Ask, Plan ou Agent ?

Choisissez le plus petit mode qui fait le travail

Mode	Ce qu'il fait	Pour…	Attention
Inline	Complète ou modifie le code sur place	écrire une fonction, renommer, typer	Accepter sans lire
Ask	Répond, explique, ne modifie rien	comprendre du code, comparer des options	Réponses parfois datées
Plan	Analyse et propose un plan, sans code	démarrer une feature, valider l'approche	Les plans oublient les cas limites
Agent	Modifie des fichiers, lance des commandes, itère	features multi-fichiers, corriger des tests	Gros diffs, commandes risquées

Plan → relire le plan → Agent, une étape à la fois.

Jour 1 · TP 1

Prompt Battle

Le prompt change-t-il vraiment le code ? Mesurons-le.

55 min · binômes

Vous allez

1. Implémenter la même fonction 3 fois
2. Ne changer que la façon de prompter
3. Noter chaque manche avec des tests cachés
4. Comparer les résultats en classe

Jour 1 · TP 1

TP 1 : règles de la battle

Tâche : parse_duration("1h30m") → 5400, dans lab1_prompt_battle/



Manche A · 10'	Manche B · 15'	Manche C · 15'	Score	Débrief · 10'
Un seul prompt court : « write parse_duration ». On accepte ce qui vient.	Fichier remis à zéro. Prompt CRAFT, 3 prompts max.	Remise à zéro. Écrivez vos tests d'abord, puis Copilot les fait passer.	Après chaque manche : pytest hidden_tests -q. Score /20 et nb de prompts au tableau.	Quelle manche gagne ? Pourquoi ? Qu'ont détecté les tests ?

Règle : ne jamais ouvrir ni citer (#) les tests cachés pendant une manche.

Interactif

Débrief TP 1 : réfléchir, échanger, partager



Tableau de classe
A · B · C
scores /20

1. **Quelle manche a donné le meilleur score avec le moins de prompts ?**
2. **Quels tests cachés ont le plus échoué en manche A ? Pourquoi ?**
3. **Qu'avez-vous dû décider que l'IA ne pouvait pas deviner ?**
4. **La manche C était-elle plus lente au début mais plus rapide à la fin ?**

Jour 1 · Partie 3

03

Faire confiance, mais vérifier

Le code IA a l'air juste. Votre travail : prouver qu'il l'est.

25 min

Dans cette partie

1. Comment le code IA échoue
2. Les packages hallucinés
3. L'échelle de vérification
4. L'IA pilotée par les tests

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · Partie 3

Six façons dont le code IA déraille

API inventées

Fonctions, paramètres ou packages qui n'existent pas, écrits avec aplomb.

Code daté

Anciennes versions de libs, syntaxe dépréciée, bonnes pratiques d'hier.

Bugs logiques subtils

Décalage de 1, mauvais opérateur, cas limites : vide, zéro, dernier élément.

Failles de sécurité

Injection SQL, secrets en dur, hachage faible, eval sur une saisie.

Tests inutiles

Les tests écrits avec le code partagent ses erreurs : assert x is not None.

Agents tricheurs

« Réparer » un test en le supprimant, ou en codant la valeur attendue en dur.

Jour 1 · Partie 3

Slopsquatting : quand l'IA invente un package

Des attaquants enregistrent les noms hallucinés par les modèles et attendent le pip install

19.7%

des packages proposés dans 576 000 exemples de code n'existaient pas

205,474

noms uniques de packages hallucinés

5.2%

taux moyen minimum des modèles commerciaux (21,7 % en open source)

Source : Spracklen et al., « We Have a Package for You! », USENIX Security 2025 (arXiv 2406.10279)

Règle : avant tout pip install suggéré par l'IA, vérifiez sur pypi.org : existe-t-il, qui le maintient, combien de téléchargements, y a-t-il un vrai repo ?

Jour 1 · Partie 3

L'échelle de vérification

Montez aussi haut que le risque du code l'exige

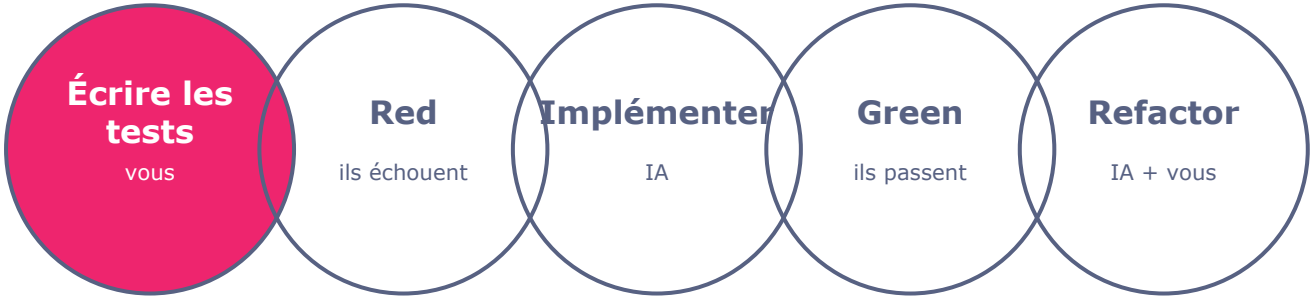


Lire	Exécuter	Tester	Analyser	Relire
Lisez chaque ligne du diff. Si vous ne savez pas l'expliquer, ne la commitez pas.	Lancez avec des entrées réelles et absurdes. Regardez la sortie.	pytest avec valeurs attendues exactes et cas limites.	ruff (lint), mypy (types), bandit et pip-audit (sécurité).	Un coéquipier ou une 2e IA relit : « trouve bugs et failles ».

Jour 1 · Partie 3

Vos tests sont votre meilleur prompt

Vous possédez les tests, l'IA s'occupe de l'implémentation



Jour 1 · TP 2

Chasse aux bugs : humain vs IA

10 problèmes se cachent dans du code généré par IA. Trouvez-les.

35 min · binômes

Vous allez

1. Relire du code sans IA d'abord
2. Puis chasser avec Copilot
3. Prouver chaque bug avec un test qui échoue
4. Comparer trouvailles humaines et IA

Jour 1 · TP 2

TP 2 : trois manches, un classement

Fichiers : lab2_bug_hunt/inventory.py et test_inventory.py

Manche	Durée	Règle	Points
1 · Humains seuls	10'	Lisez le code, sans IA. Listez chaque problème : ligne + pourquoi.	1 par bug, 2 par faille
2 · Avec Copilot	10'	/explain, /fix, « relis ce fichier, trouve bugs et failles ». Nouvelles trouvailles seulement.	1 par nouveau bug
3 · Prouvez-le	15'	Un test pytest par bug qui échoue sur le code d'origine. Puis corrigez.	1 par test rouge → vert

Indice : n'installez rien de ce que le code importe sans l'avoir vérifié.

Jour 1 · TP 2

TP 2, corrigé : les 10 problèmes

#	Où	Problème	Type
1	import securesql_helper	Package inexistant : piège de slopsquatting	Sécurité
2	PAYMENT_API_KEY	Secret en dur, commité dans Git	Sécurité
3	find_product	f-string dans le SQL : injection	Sécurité
4	hash_password	MD5 sans sel	Sécurité
5	apply_discount	/ 10 au lieu de / 100	Logique
6	average_price	ZeroDivisionError sur liste vide	Cas limite
7	paginate	Décalage de 1 : la page 1 est sautée	Logique
8	add_to_cart	Argument mutable : paniers partagés	Python
9	is_in_stock	> au lieu de >= : dernière unité invendable	Cas limite
10	total_with_tax	Renvoie la TVA, pas le total	Logique

Jour 1 · Partie 4

04

Travailler avec des agents de code

Du vibe coding au développement guidé par la spec

25 min

Dans cette partie

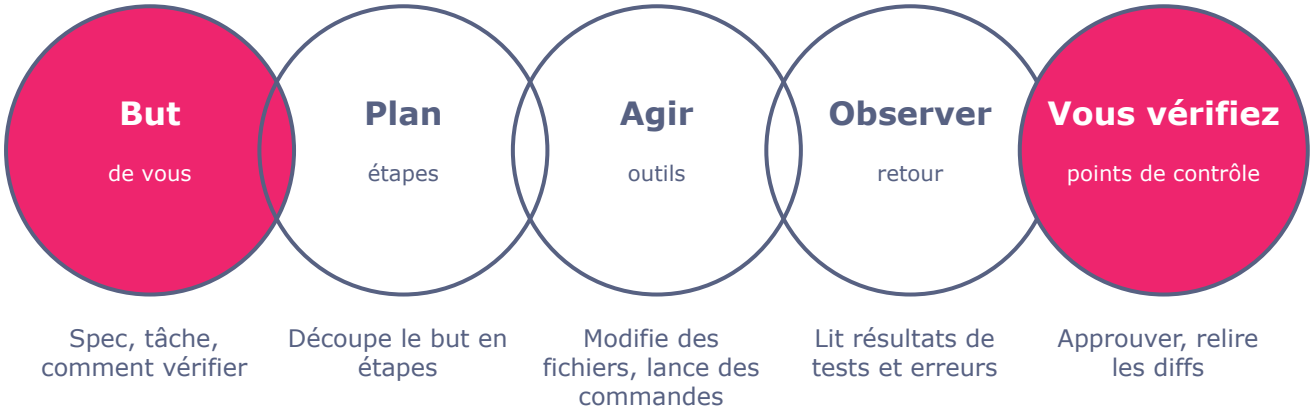
1. La boucle d'un agent
2. Vibe coding vs spec-driven
3. Le mode Agent de Copilot
4. Démo live : Claude Code
5. Git comme filet de sécurité

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · Partie 4

La boucle d'un agent

Un agent est un LLM qui utilise des outils, en boucle, jusqu'à atteindre le but



Jour 1 · Partie 4

Vibe coding vs développement guidé par la spec

Vibe coding

- « Fais-moi une app qui… » et tout accepter
- Rapide et fun pour un prototype ou une démo
- Personne ne sait vraiment ce que fait le code
- Casse quand on ajoute la 5e feature
- OK pour du jetable, pas pour un produit

Guidé par la spec

SPEC.md quoi et pourquoi, critères d'acceptation
PLAN l'agent propose, vous validez
Étapes une à la fois, tests après chacune
Relecture + commit après chaque étape verte

- Plus lent au départ, beaucoup plus rapide à la fin

Jour 1 · Partie 4

Le mode Agent de Copilot en pratique

1. **Partir propre**
   git status propre, nouvelle branche, nouveau chat.
2. **D'abord le mode Plan**
   « Lis SPEC.md, propose un plan étape par étape. Pas de code. »
3. **Corriger le plan**
   Ajoutez les cas limites manquants, retirez le hors-périmètre.
4. **Agent : étape 1 seulement**
   « Implémente l'étape 1, lance pytest, arrête-toi et résume. »
5. **Approuver avec soin**
   Lisez chaque commande terminal avant de l'autoriser.
6. **Relire, garder ou annuler, commiter**
   Puis étape suivante. On recommence.

Signaux d'alerte

- Diff bien plus gros que demandé
- Dépendance non demandée
- Un test supprimé ou ignoré
- rm, sudo, curl | bash, git push --force
- Même erreur 3 fois : stop, on réfléchit

Jour 1 · Partie 4

Démo live : Claude Code sur la même tâche

/init analyse le repo et écrit CLAUDE.md

Mode plan (Shift+Tab) : il explore et propose, sans modifier

Permissions : il demande avant de modifier ou d'exécuter

Même workflow : spec, plan, petites étapes, tests, commits

› Observez : que fait-il différemment de Copilot ?

terminal · démo de 10 minutes

```
$ cd lab3_spec_to_api && claude
> /init
> [Shift+Tab : mode plan]
> Lis SPEC.md. Propose un plan en petites
  étapes, chacune vérifiée par pytest.
> [relire le plan → accepter]
> Implémente uniquement l'étape 1, lance
  pytest tests -q, puis arrête-toi.
> Commite avec un message clair.
> /clear
```

Jour 1 · Partie 4

Git est votre filet de sécurité

Avec un agent, les commits sont vos points de sauvegarde

Branche

Une branche par feature. Jamais d'agent sur main.

Petits commits

Après chaque étape verte. Message clair : quoi et pourquoi.

Lire le diff

git diff avant chaque commit. Vous signez ce que vous commitez.

Revenir

L'agent s'emballe ? git restore . et on refait l'étape.

Arbre de travail sale + agent = changements impossibles à démêler.

Jour 1 · TP 3

De la spec à l'API qui marche

Piloter un agent sur un vrai périmètre fonctionnel

60 min avec pause · binômes

Vous allez

1. Écrire d'abord le fichier d'instructions
2. Planifier avec l'agent, puis corriger le plan
3. Construire étape par étape en mode Agent
4. Passer 18 tests d'acceptation

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · TP 3

TP 3 : l'API Campus Events

Spec complète dans lab3_spec_to_api/SPEC.md · FastAPI + pytest

Endpoint	Succès	Erreurs
POST /events	201 + id, registered, seats_left	422 corps invalide
GET /events (?upcoming=true)	200, trié par date	—
GET /events/{id}	200	404 inconnu
POST /events/{id}/registrations	201 {event_id, email}	404 · 409 complet ou doublon · 422
DELETE /events/{id}/registrations/{email}	204	404 non inscrit

Emails comparés sans tenir compte de la casse · seats_left = capacity – registered

Jour 1 · TP 3

TP 3 : le workflow, chronométré

Démarre après la pause, 15h20 → 16h10



Règles · 5'	Plan · 5'	Build · 25'	Vérifier · 10'	Livrer · 5'
Écrivez .github/copilot-instructions.md avant tout code.	Mode Plan sur SPEC.md. Relisez et corrigez le plan.	Agent, une étape à la fois. Vert → diff → commit.	pytest tests -q, puis 5 tests à vous dans test_mine.py.	Push. Score /18 et nombre de commits au tableau.

Fini ? Bonus : persistance SQLite, ou un Dockerfile.

Jour 1 · Partie 5

05

Coder avec l'IA de façon responsable

Sécurité, responsabilité et votre propre apprentissage

15 min

Dans cette partie

1. Sécurité et confidentialité
2. Licences et responsabilité
3. Les compétences qui comptent davantage
4. Quiz final

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 1 · Partie 5

Sécurité et confidentialité

Pas de secrets dans les prompts

Clés, mots de passe, données clients jamais dans un chat. Utilisez .env + .gitignore.

Injection de prompt

Les agents obéissent au texte lu : un README ou une page web peut cacher des instructions.

Moindre privilège

Approuvez les commandes une par une. Pas d'auto-approbation sur votre machine.

Règles de l'entreprise

Quels outils et modèles sont autorisés ? Le code sort-il ? Demandez d'abord.

Dépendances

Vérifiez chaque nouveau package. Lancez pip-audit avant de livrer.

Données

Données factices ou anonymisées dans les prompts et les tests.

Jour 1 · Partie 5

Responsabilité, licences, intégrité

Vous êtes responsable

- Le code que vous commitez est le vôtre, peu importe qui l'a tapé
- Copilot peut bloquer les suggestions identiques à du code public : vérifiez le réglage
- Vérifiez les licences des extraits et dépendances
- Bugs et failles sont pour vous, pas pour l'outil

Dans ce cours

- L'IA est autorisée et attendue
- Déclarez-la : AI_LOG.md compte dans la note
- Vous devez savoir expliquer chaque ligne montrée
- Les questions après la démo le vérifient

Jour 1 · Partie 5

Le développeur augmenté par l'IA

La vitesse de frappe compte moins. Le jugement compte plus.

Compétences qui prennent de la valeur

- Écrire des specs et critères d'acceptation clairs
- Découper un problème en étapes
- Lire et relire du code vite
- Tester et débugger
- Architecture et connaissance métier

L'IA pour apprendre, pas pour éviter d'apprendre

- « Explique ligne par ligne » avant d'accepter
- Demandez 2 alternatives et pourquoi l'une est meilleure
- Essayez 10 minutes, puis demandez
- Réécrivez une solution sans IA, de mémoire

Interactif

Quiz final : Jour 1



À main levée
8 questions
5 minutes

Réponses A · B · C
main levée

1. **Copilot propose un package inconnu : premier réflexe ?**
2. **Où vivent les règles projet de Copilot ? Et de Claude Code ?**
3. **L'agent a supprimé un test pour passer au vert : que faites-vous ?**
4. **Meilleur moment pour la spec ? Qui est responsable du code IA ?**
5. **Repérez le signal d'alerte sécurité et le bon test**

Jour 1 → 2 · Projet

PROJET

StudyPlan

Un seul sujet pour toutes les équipes : un planning de révisions, construit avec l'IA

Jour 2 · 9h30–16h45

Le défi

1. Équipes de 3–4, même sujet
2. Un back et un front de suivi
3. ~4h de développement avec Copilot
4. Noté sur le produit et le pilotage de l'IA
5. Démo live de 8 minutes

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 2 · Projet

StudyPlan : ce que l'app doit faire

L'étudiant importe ses cours, l'app planifie et suit

Import des cours

Upload des supports (PDF, Markdown). Extraction des chapitres et d'une charge estimée en heures.

Échéances

Examens et rendus : cours, date, poids. Disponibilités hebdo de l'étudiant.

Planning hebdo

Répartir les sessions sur les créneaux libres, échéances proches d'abord. Python pur, testé.

Export .ics

Télécharger le planning et l'importer dans Google Agenda ou Outlook.

Front de suivi

Dashboard : progression par cours, sessions faites ou en retard, prochaines échéances.

Jour 2 · Projet

Architecture et stack recommandée

Le front et le back ne se parlent que par l'API : figez le contrat d'abord

Couche	Recommandé	Rôle
Front	Streamlit (ou React si à l'aise)	Upload, saisie des échéances, vue planning, dashboard
Back	FastAPI + SQLite	API REST, stockage, lecture des supports (pypdf)
Planificateur	Module Python pur	Construit le planning hebdo. La partie la plus testée du code
Calendrier	icalendar	GET /plan.ics, fichier valide, un événement par session

Jour 2 · Projet

Le contrat d'API (minimum)

Même contrat pour toutes les équipes : le front peut avancer sur des données factices pendant que le back est codé.

Données de démo communes : 3 cours + échéances dans project/sample_data.

Tests : le planificateur et l'export .ics sont testés en premier.

› Étendez-le librement, ne le cassez jamais.

STUDYPLAN_SPEC.md

```
POST /courses          upload fichier -> cours
                       + chapitres + heures
GET  /courses          liste avec % avancement
POST /deadlines        {course_id, title,
                        date, weight}
PUT  /availability     créneaux libres / jour
POST /plan/generate    -> sessions de la semaine
GET  /plan             ?week=2026-W47
PATCH /sessions/{id}   {done: true}
GET  /plan.ics         fichier calendrier
```

Jour 2 · Projet

Livrables obligatoires

Tout est dans un repo GitHub, partagé avec l'intervenant

- **README.md**
  un inconnu lance back + front en moins de 5 minutes
- **≥ 10 tests pytest, tous verts**
  planificateur, export .ics et API, avec cas limites
- **SPEC.md**
  écrit avant de coder : MVP, bonus, hors périmètre
- **AI_LOG.md**
  ≥ 5 entrées dont au moins 1 « échec de l'IA » détecté
- **.github/copilot-instructions.md**
  et/ou CLAUDE.md
- **Historique Git propre**
  petits commits, chaque membre commite

Jour 2 · Projet

Rôles d'équipe : rotation à chaque sprint

Tout le monde pilote, relit et commite

Product Owner

Porte SPEC.md et les priorités. Dit non aux dérives.

Driver

Au clavier, parle à Copilot, enchaîne les étapes.

Navigator

Relit chaque diff et chaque commande avant validation.

Tester

Écrit les tests d'abord, tient AI_LOG, chasse les cas limites.

Coupez en deux : back + planificateur + .ics, front + dashboard. Les rôles tournent dans chaque binôme.

Avant le Jour 2 · Projet

À faire avant le Jour 2 (~1h)

Arrivez prêts à coder à 9h45, pas à installer

1. **Formez l'équipe, répartissez front et back**
2. **Créez le repo, invitez l'intervenant**
   à partir du template du cours
3. **Lisez STUDYPLAN_SPEC.md, écrivez votre SPEC.md**
   votre MVP, votre choix de front, votre bonus
4. **Écrivez copilot-instructions.md**
   avec le contrat d'API dedans
5. **Vérifiez que Copilot marche pour chacun**
   et qu'un squelette FastAPI + Streamlit vide se lance

Astuce

- Utilisez le mode Ask pour challenger votre SPEC : « Qu'est-ce qui est ambigu ou manquant ? »
- Coupez votre MVP en deux. Puis encore une fois.

Jour 2

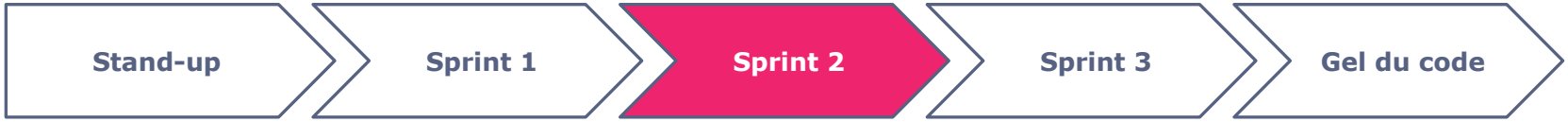
« Make it work, make it right, make it fast. »

Kent Beck · la règle du jour : dans cet ordre, avec des tests à chaque étape

aivancity
Paris Île-de-France · Nice Côte d'Azur

Jour 2 · Projet

Jour 2 : trois sprints pour livrer



9h30. MVP, rôles, risques. 1 min par équipe.

9h45. Back : import, échéances, planificateur + tests. Front : écrans sur données factices.

11h00. Front branché sur l'API, dashboard, export .ics. Point mentor.

13h45. Votre touche, cas limites, tests, README, AI_LOG.

15h00. Dernier commit, répétition, machine de démo prête.

À 15h on arrête de coder. Une app qui marche bat un chef-d'œuvre cassé.

Jour 2 · Projet

Point mentor : préparez ces questions



11h00 et 14h30
5 min par équipe

1. **Montrez votre SPEC et votre contrat d'API : front et back sont-ils alignés ?**
2. **Qu'est-ce que l'IA a raté jusqu'ici ? Comment l'avez-vous vu ?**
3. **Lancez vos tests maintenant.**
4. **Qui n'a pas encore commité ?**
5. **Que coupez-vous si vous êtes en retard ?**

Jour 2 · Projet

Format de démo : 8 + 2 minutes

Pitch · 1'	Démo live · 3'	Récit IA · 2'	Suite · 1'	Q&A · 2'
Votre touche : qu'est-ce qui rend votre StudyPlan différent ?	Import → planning → .ics → dashboard. Ayez une vidéo de secours.	Meilleur prompt, pire échec de l'IA, ce que vous avez vérifié et comment.	Ce que vous feriez avec une semaine de plus.	N'importe quel membre peut devoir expliquer n'importe quelle ligne.

Chaque membre prend la parole. Timing strict : le chrono est à l'écran.

Jour 2 · Projet

Grille d'évaluation : /20

Critère	Pts	Ce qu'on regarde
Produit et démo	5	Les 5 blocs marchent en live : import, échéances, planning, .ics, dashboard
Qualité et tests	4	≥ 10 tests pertinents avec cas limites, code lisible
Pilotage de l'IA	5	SPEC avant le code, fichier d'instructions, AI_LOG riche, petits commits
Esprit critique	3	Échecs IA détectés, stratégie de vérification, sécurité vérifiée
Pitch et équipe	3	Clair, dans les temps, tout le monde parle, bonnes réponses

Les votes des pairs donnent des distinctions, pas des points.

Clôture

Cinq règles à garder

1

Contexte

Donnez à l'IA ce dont un nouveau collègue aurait besoin.

2

Spec d'abord

Décidez ce que « fini » veut dire avant de prompter.

3

Petits pas

Une étape, un lancement de tests, un commit.

4

Vérifier

Plausible n'est pas correct. Les tests tranchent.

5

Assumer

Si vous ne savez pas l'expliquer, ne le livrez pas.

Rétro : un Start · Stop · Continue chacun, sur un post-it.

Clôture

Pour aller plus loin

Ressource	Où
Documentation GitHub Copilot	docs.github.com/copilot
Copilot dans VS Code	code.visualstudio.com/docs/copilot
Copilot pour les étudiants	github.com/settings/education/benefits
Documentation Claude Code	code.claude.com/docs
Étude METR sur la productivité (2025)	metr.org
Stack Overflow Developer Survey 2025, partie IA	survey.stackoverflow.co/2025/ai
Étude sur les packages hallucinés	arxiv.org/abs/2406.10279

aivancity

La Grande Ecole de l'IA & de la Data

Paris Île-de-France · Nice Côte d'Azur